SMART WASTE RECYCLING SYSTEM

Assignment 2 — System Design Document

Purpose	Define system capabilities, service responsibilities, API contracts, and database design.
Backend	Node.js, Express.js, MySQL, mysql2
API style	REST over HTTP; JSON request and response bodies
Document status	Design baseline; implementation status is stated explicitly in this document.

Prepared for project review and implementation planning

1. Project Overview

The Smart Waste Recycling System is intended to help users submit recyclable-waste pickup requests and allow the application to store and manage waste categories and pickup records. The current backend is a single Express application connected to a MySQL database. Waste and pickup responsibilities are separated logically by API routes; they are not currently separate deployed microservices.

2. System Capabilities

ID	Capability	Description	Status
C1	Browse waste categories	Return available categories from MySQL.	Implemented
C2	Create pickup request	Store user, category, quantity, and address.	Implemented
C3	View pickup requests	Return the stored pickup request list.	Implemented
C4	View one pickup	Retrieve one request by ID.	Planned
C5	Update pickup status	Change status for an eligible request.	Planned
C6	Cancel pickup	Cancel an eligible request.	Planned

3. Service Architecture

The client sends HTTP requests to the Express REST API. Route handlers use the mysql2 driver to query or update the MySQL database. The logical service responsibilities are:

Component	Responsibility	Current endpoint(s)
Waste Category routes	Read available waste categories.	GET /api/waste-categories
Pickup routes	Create and list pickup requests.	POST /api/pickups GET /api/pickups
Planned pickup operations	Retrieve, update, or cancel one pickup.	GET /api/pickups/{id} PATCH /api/pickups/{id}
MySQL database	Persist categories and pickup requests.	waste_categories, pickups

Request flow (logical view)

```
Client
  | HTTP request / JSON
  v
Express REST API
  | route handler + SQL query
  v
MySQL database
  | query result
  v
Express REST API -- JSON response --> Client
```

4. Service Contracts

4.1 Get waste categories

Method: GET **Path:** /api/waste-categories
Request body: None
Success: 200 OK, application/json
Failure: 500 Internal Server Error if the database query fails.

```
{
  "success": true,
```

```
  "count": 4,
  "categories": [
    {"id": 1, "name": "plastic"},
    {"id": 2, "name": "paper"},
    {"id": 3, "name": "metal"},
    {"id": 4, "name": "glass"}
  ]
}
```

4.2 Create a pickup request

Method: POST **Path:** /api/pickups
Header: Content-Type: application/json
Success: 201 Created
Current failure handling: 500 Internal Server Error if the database insert fails. Request-body validation and consistent client-error responses are not yet implemented.

```
{
  "userId": 1,
  "wasteCategory": 1,
  "quantityKg": 3,
  "address": "Hostel Block A"
}
```

4.3 Get all pickup requests

Method: GET **Path:** /api/pickups
Request body: None
Success: 200 OK with a JSON object containing success, count, and pickups.
Failure: 500 Internal Server Error if the database query fails.

4.4 Planned contracts

Operation	Method and path	Intended behavior
Get one pickup	GET /api/pickups/{id}	Return a pickup by ID; 404 if not found.
Update status	PATCH /api/pickups/{id}	Update an allowed status; validate input and existence.
Cancel pickup	DELETE /api/pickups/{id}	Cancel an eligible pickup; define behavior for missing/already-

5. Database Schema

Table	Column	Type / constraint	Meaning
waste_categories	id	INT, PK, AUTO_INCREMENT	Unique category identifier
waste_categories	name	VARCHAR(50), NOT NULL, UNIQUE	Category name
pickups	id	INT, PK, AUTO_INCREMENT	Unique pickup identifier
pickups	user_id	INT, NOT NULL	Requesting user's identifier; no users
pickups	waste_category_id	INT, FK, NOT NULL	References waste_categories.id
pickups	quantity_kg	DECIMAL(8,2), NOT NULL	Waste quantity in kilograms
pickups	address	VARCHAR(255), NOT NULL	Pickup location
pickups	status	VARCHAR(30), NOT NULL, default 'Pending'	Pickup request status
pickups	created_at	TIMESTAMP, default current timestamp	Creation time

Relationship: one waste category may be referenced by many pickup requests. Each pickup row references one waste category through pickups.waste_category_id. The current schema does not include a users table, so user_id is not currently a foreign key.

6. Architecture Validation

Requirement	Design response	Status / gap
Browse categories	GET route queries MySQL.	Implemented and tested with HTTP 200.
Persist pickup requests	POST route inserts a row using parameterized SQL.	Implemented; request validation is a gap.
List pickup requests	GET route selects records from MySQL.	Implemented; test coverage should be docum
Retrieve/update/cancel one pickup	Planned resource-oriented endpoints.	Not implemented yet.
Database integrity	PKs, unique category names, category FK.	Implemented; no users table/FK.
Reliable error contract	JSON errors for DB failures.	Needs input validation and consistent error sh

7. Design Decisions and Next Improvements

• Keep the first implementation as one Express application with logically separated route responsibilities.

• Use parameterized SQL queries to avoid concatenating untrusted input into SQL statements.

• Add request validation for userId, wasteCategory, quantityKg, and address before inserting a pickup.

• Add GET-by-ID, PATCH, and DELETE pickup endpoints with clear not-found and validation behavior.

• Introduce a users table and a foreign key for pickups.user_id when user management is in scope.

• Constrain valid pickup status values and define permitted state transitions.

• Keep documentation aligned with implementation; mark unfinished operations as planned until tested.

8. Current Implementation Summary

At the time this document was prepared, the home route, GET /api/waste-categories, POST /api/pickups, and GET /api/pickups are implemented. The waste-category endpoint was tested and returned HTTP 200 with four categories from MySQL. The GET-by-ID, PATCH, and DELETE pickup operations remain planned. The SQL schema file has been reviewed but has not yet been verified by recreating the database.

This document describes the current project state and proposed design. It should be updated as endpoints are implemented and tested.